



CSM Coding Standards

ACI Internal Use Only

March 18, 2005

© 2005 by ACI Worldwide Inc.

All rights reserved. All information contained in this document is confidential and proprietary to ACI Worldwide Inc. This material is a trade secret and its confidentiality is strictly maintained. Use of any copyright notice does not imply unrestricted or public access to these materials. No part of this document may be photocopied, electronically transferred, modified, or reproduced in any manner without the prior written consent from:

ACI Worldwide Inc.
330 South 108th Avenue
Omaha, Nebraska 68154
(402) 390-7600

BASE24, BASE24-atm, BASE24-billpay, BASE24-from host maintenance, BASE24-pos, BASE24 Remote Banking, BASE24-telebanking, BASE24-teller, and XPNET are trademarks or registered trademarks of ACI Worldwide Inc., Transaction Systems Architects, Inc., or their subsidiaries.

Other companies' trademarks, service marks, or registered trademarks and service marks are trademarks, service marks, or registered trademarks and service marks of their respective companies.



TABLE OF CONTENTS

1. DOCUMENT CONTROL.....	1
2. SCOPE.....	2
3. NAMING CONVENTIONS	2
3.1 SUBVOL NAMES.....	2
3.2 FILE NAMES	3
4. CODING STANDARDS	4
4.1 VANILLA SOURCE CODE	4
4.2 MODIFYING COBOL MODULES	4
4.3 MODIFYING TAL MODULES.....	4
4.4 CSM HISTORY SECTION.....	5
4.5 DOCUMENTING CSM CODE CHANGES.....	6
4.6 DDLs	10
4.7 PATHWAY SCREENS	10
4.8 DEVICE CONFIGURATION FILES	11
4.9 COBNAMQS FILE	11
4.10 DHNAMEQS FILE	11
4.11 UTILITIES	11
4.12 DUMMY PROCEDURES.....	12
4.13 LCONF ASSIGNS AND PARAMS.....	12
4.14 LOG MESSAGES.....	12
4.15 TOKEN IDS	12
4.16 FIELD/VARIABLE INITIALIZATION	12
4.17 CSM MAINTENANCE	13
5. CSM TESTING STANDARDS	13
5.1 CSM UNIT TESTING.....	13



5.2	CSM REGRESSION TESTING.....	14
5.3	CSM TEST SCRIPTS	14
6.	CSM MACROS AND BUILDSKEL6	14
6.1	USING BUILDSKEL6	15
7.	CSM MAKE STANDARDS	16
8.	TAL EXAMPLE CODE.....	16



1. DOCUMENT CONTROL

<u>VERSION:</u>	<u>AUTHOR'S INITIALS:</u>	<u>DATE DISTRIBUTED:</u>
E	Misc.	1998/12/14
MAJOR CHANGES:	Document upgraded to support Word 2000	
F	MSA	2000/07/25
MAJOR CHANGES:	Document upgraded to support Word 2000	
G	KLH	2000/09/14
MAJOR CHANGES:	Document upgraded to convert RPQ to CSM	
H	TSB	2005/03/18
MAJOR CHANGES:	Modified and renamed Section 4.5 Documenting CSM Code Modifications, to include developer comments. Also, made this section a more step-by-step description of documenting a CSM. Modified Section 4.14 Log Messages to include a statement about the text of the message. Added Sections 4.1 Vanilla Source Code, 4.16 Field/Variable Initialization and 4.17 CSM Maintenance. Added Sections 5 CSM Testing Standards, 5.1 CSM Unit Testing, 5.2 CSM Regression Testing, 5.3 CSM Test Scripts, 6 CSM MACROS and BUILDSKEL6, 6.1 Using BUILDSKEL6, and 7 CSM Make Standards. Other modifications were made: to remove references to 5.x or were cosmetic in nature.	



2. SCOPE

This document contains ACI's CSM development standards for BASE24 'Classic' code (atm, pos, fhm, teller, etc.).

3. NAMING CONVENTIONS

3.1 SUBVOL NAMES

The CSM subvol name will be derived as follows:

<customer acronym><product id>

where:

<customer acronym> is the 4 character identifier assigned for the customer. Customers with an acronym, which is less than 4 characters, will have the acronym padded with zeroes. (Examples: "GW00", "FCC0", "BNK1", etc.).

<product id> is the 4 character product identifier (Examples: AT60, PS60, FH60, etc.).

The exception to this rule is the Switch Interface modules. The subvol for Switch Interfaces will be derived as follows:

<customer acronym><Switch Interface acronym>

where:

<customer acronym> Same as above.

<Switch Interface acronym> is the assigned product identifier of the Switch Interface (Examples: "MDS", "VISA", "STAR", etc.).

Note that the Switch Interface Modules must be treated differently because they currently share common naming conventions for their DDLs, Requestors, etc.

For Customer (acronym CUST) the following would be some typical subvols:



CUSTBA60 - Rel 6.0 Base code
CUSTAT60 - Rel 6.0 ATM code
CUSTPS60 - Rel 6.0 POS code
CUSTBIC - BIC Switch Interface code
CUSTPLUS - Plus Switch Interface code

Note: For those modules that span multiple product subvols (i.e., HISF, HISO, REFR, EXTR, etc.), there would be a subvol for each product. For example, if a CSM to ISO Host Interface affected Base, ATM, and POS, the appropriate files would be on the CUSTBA60, CUSTAT60, and CUSTPS60 subvols.

3.2 FILE NAMES

The letter 'Q' will be added to each CSM'd module before the 'type' suffix to indicate that the module is CSM'd. In some cases adding the 'Q' to the filename will require the original filename to be abbreviated (e.g. AUTHLQS). A few examples will illustrate the concept:

C910QS	- C910 Device Handler CSM source file
C910QC	- C910 Device Handler CSM compile file
C910Q	- C910 Device Handler CSM object file
AUTHQS	- Auth CSM source file
AUTHLBQS	- Authlibs CSM source file
AUTHQC	- Auth CSM compile file
AUTHQB	- Auth CSM bind file
AUTHQ	- Auth CSM object file
RQTDFQS	- ATM TDF Requester source file
RQTDFQC	- ATM TDF Requester compile file
SVTDFQS	- ATM TDF Server source file
SVTDFQC	- ATM TDF Server compile file
SVTDFQ	- ATM TDF Server object file
COBNAMQS	- CSM'd COBOL tables
DHNAMEQS	- CSM'd Device Handler tables
BAHISOQD	- CSM'd Base ISO Host Interface Dummy Procedures
BAUTILQS	- Base Utilities source file
BAUTILQC	- Base Utilities compile file
BAUTILQ	- Base Utilities object file

New modules developed by the CSM group will also include the letter 'Q' before the 'type' suffix (e.g. NEWDMQS).

Refer to the CSM Make Standards 2000 1.5 Document for file naming conventions.



4. CODING STANDARDS

4.1 *Vanilla Source Code*

Because of rolling code, the developer should copy the vanilla source code for any module being modified to a customer specific vanilla subvol.

In the case where the CSMs are being developed for an existing customer and the customer's fix level is different than the current product version, then the vanilla code is taken from the customer and stored on the customer specific vanilla subvol.

The CSM subvol name will be derived as follows:

<customer acronym>VANL

The make stream used, while developing, a CSM will point at the customer specific vanilla subvol, rather than the current product version.

4.2 *Modifying COBOL Modules*

For modules written in COBOL, the vanilla file will be copied to a new file on the customer specific subvol (refer to the naming conventions described in the previous section). All modifications will then be applied to the new file. Note that the original vanilla file will remain unaltered.

4.3 *Modifying TAL Modules*

For modules written in TAL, the developer should create a new file using the naming conventions described in the previous section. This file should only contain those procedures and sections that are to be modified by the CSM(s). Procedures and program sections (e.g., globals, DDLs, Externals and Forwards, etc.) not altered by the CSM(s) will be sourced-in using the "?SOURCE" statement. The developer should remember to remove the ?SOURCE statement for all of the vanilla procedures that are included as part of the CSM code.

The latest FIXER line from the vanilla source file will be copied to the first line of the CSM source file when CSM development is begun. This way, the FIX level of vanilla when the CSM was coded can be established.

The TACL macro BUILDSKEL6 should be used to create the CSM source file. See Section 13 for further BUILDSKEL6 information.



Refer to the end of this document for an example of TAL CSM Code.

Following are some rare, unique situations which call for special handling:

One situation is a new Custom Interface, which has new DDLs in addition to the SWCHTAL (Generic Switch) DDLs and does not use any source code from the Generic Switch. It is recommended that all related Generic Switch DDLs be moved into the Custom Interface DDLs.

Another is when a MAIN procedure is modified. Since the vanilla MAIN procedure name changes whenever a fix is applied to the vanilla code and the CSM'd MAIN procedure name may not be changed, a bind using the vanilla object code will fail. Having two MAIN procedures declared, one from the CSM module and one from the vanilla module caused this failure. This scenario occurs when the CSM'd module sources and binds from two different modules. This may involve a switch that also sources and binds from the Generic Switch as well as from its vanilla interface. It is recommended that all procedures from the vanilla or Generic Switch, both source and externals, except those being CSM'd, be sourced into the CSM'd module. The bind command in the make file is then modified to use the newly created csm object instead of the Generic Interface object. BUILDSKEL6 can be used with the Generic Switch to create sections of the source procedures. This is similar to the standard used in CSMing the Authorization Process.

When a switch specific module requires a CSM to a Generic Switch procedure, it is recommended that the procedure to be modified be brought into the CSM'd switch specific code rather than creating a SWCHQS module.

4.4 CSM History Section

The following illustrates the format and content of the CSM History Section. This is to be added to EVERY source file CSM'd **before** the vanilla history section for those modules that are copied in their entirety from vanilla (COBOL, SCOBOL, DDL, etc. files). For TAL modules, since we only have the affected procs in the CSM source file, the vanilla History section should be omitted since the vanilla FIXER line is deemed to be enough reference.

New modules developed by the CSM group will use the CSM History Section instead of the Standard Product History Section. The only difference is that the 'PROCS MODIFIED' portion will not be included.



```
#####!  
!#   CSM History:                                     #!  
!#                                     #!  
!#   CSM:      Description:                           #!  
!#   -----  
!#   <date>      CSMS to <module> ver. n.n.nn  <dev inits>      #!  
!#   CSM nn      <Description from CSM request form>           #!  
!#   Summary:    <Functional description of CSM>               #!  
!#               <Impact on this module>                       #!  
!#                                     #!  
!#               PROCS MODIFIED:  <proc 1 name>                #!  
!#                                   <proc 2 name>                #!  
!#                                   <proc n name>                #!  
!#                                     #!  
#####!
```

If a fix (not an add-on CSM) were made to an existing CSM then the history section would be similar to the one that would be written for a vanilla fix:

```
#####!  
!  
!#   < Date>      <Developer inits>                                     #!  
!#   SYMPTOM:    <Symptom of the problem>                             #!  
!#   PROBLEM:    <Description of the actual problem>                 #!  
!#   FIX:        <Description of modification to fix the problem>    #!  
!#               PROCS MODIFIED:  <proc 1 name>                     #!  
!#                                   <proc 2 name>                     #!  
!#                                   <proc n name>                     #!  
!#   DEPENDENCIES:  <Implementation Dependencies>                   #!  
!#   REFERENCE:    <RPC or RCC number>                               #!  
!#               #####!  
!
```

For COBOL source files, the "!" TAL comment character will be replaced with the "*" COBOL comment character in the examples.

4.5 Documenting CSM Code Changes

VANILLA CODE WILL NOT BE UNSEQUENCED!!!!!! TAL CSM source files will only contain the procedures modified; all other modules are duplicated in their entirety from the vanilla source file. New modules developed by the CSM group will not include sequence numbers.

Vanilla code that is no longer needed can be commented out or deleted. It is preferable to comment out three or four lines of vanilla code that are no longer needed and to delete unneeded vanilla code that contains more than three or four lines. The programmer should use common sense in determining which method is better for the situation. The “–” (two dashes) is the preferred method for commenting out an entire line of vanilla code.

If a vanilla line is changed, the sequence number should be removed.

Below is an example of how code should be modified:

Vanilla Code:

```
begin                                     !03104
mtp[ msg^length - 1 ] := null;          !03105
@ptr := @mtp;                          !03106
scan^fs^d( ptr );                      !03107
atm.n50.luno ':= ' ptr[1] for $len( atm.n50.luno ); !03108
end;                                    !03109
```

CSM Coded by Developer:

```
begin                                     !03104
If tdf.n50.term^proto = snax^hls^protocol^d then
    mtp[msg^length] := null
else
    mtp[msg^length - 1] := null;
@ptr := @mtp;                          !03106
scan^fs^d( ptr );                      !03107
atm.n50.luno ':= ' ptr[1] for $len( atm.n50.luno ); !03108
end;                                    !03109
```

CSM code will be bracketed with the appropriate BEGIN/END comments. The format of these comments is as follows:



```
!*****!  
!* BEGIN CSM #nn - <Description from CSM request form> - <inits>  !  
!-----!  
:  
:  
:  
!-----!  
!*   END CSM #nn - <Description from CSM request form> - <inits>  !  
!*****!
```

An example of CSM Code using the required bracket comments is as follows:

```

begin                                                    !03104
!*****!
!* BEGIN CSM #01 - SNAX/HLS Support - JDD                !
!-----!

    If tdf.n50.term^proto = snax^hls^protocol^d then
        mtp[msg^length] := null
    else
        mtp[msg^length - 1] := null;
!-----!
!* END CSM #01 - SNAX/HLS Support - JDD                  !
!*****!
    @ptr := @mtp;                                       !03106
    scan^fs^d( ptr );                                   !03107
    atm.n50.luno ':= ' ptr[1] for $len( atm.n50.luno ); !03108
end;                                                    !03109

```

As CSM code must be maintained and may be built on in future CSMs, the developer should include comments that describe the functionality being implemented. The developer should use common sense in determining where comments would be appropriate. An example of this type of comment is included below:

```

begin                                                    !03104
!*****!
!* BEGIN CSM #01 - SNAX/HLS Support - JDD                !
!-----!
! JDD - THE VANILLA CODE REPLACED THE TRAILING ETX WITH A
! NULL TO ENSURE SCANS STOPPED. SNA DOESN'T
! HAVE THE ETX CHARACTER SO WE HAVE TO PLACE THE
! SCAN STOPPER AT MSG^LENGTH INSTEAD OF
! MSG^LENGTH - 1.
!
    If tdf.n50.term^proto = snax^hls^protocol^d then
        mtp[msg^length] := null
    else
        mtp[msg^length - 1] := null;
!-----!
!* END CSM #01 - SNAX/HLS Support - JDD                  !
!*****!
    @ptr := @mtp;                                       !03106
    scan^fs^d( ptr );                                   !03107
    atm.n50.luno ':= ' ptr[1] for $len( atm.n50.luno ); !03108
end;                                                    !03109

```

4.6 DDLs

If a DDL has to be CSM'd, then it should be copied to the appropriate CSM subvol. The DDLS file will also have to be copied. The ?SOURCE statements in the DDLS file will have to be modified so that the unchanged DDLs are retrieved from the vanilla vol/subvol. The DDLS file will be sent to the customer along with instructions to modify the VOLUME references in the ?SOURCE statements. While this method will require additional disk space; it will minimize the impact to TAL source code because of the use of SSV ASSIGNS. COBOL COPY statements may have to have SUBVOL references changed to use the CSM'd DDLs.

If a choice exists between modifying a DDL and using a DEFINE to point to a currently unused area of a current DDL (i.e., a User Field), the DDL should be modified. While this will require more work on the developer and customer's part, the increased visibility of the change outweighs such considerations. The customer will be aware of the fact that the DDL is modified and will be able to judge the impact of future fixes to the DDL.

If a custom DDL must be created, the DDL source file should be named: DDLQxxxx (where xxxx is some appropriate acronym). Custom DDLs should be compiled into the appropriate vanilla DDL file (e.g., BADDLTAL, ATDDLTAL, etc.).

4.7 Pathway Screens

When a PATHWAY Screen is CSM'd, only the affected Screen will be sent to the customer. The following naming convention has been adopted:

<product id>SCRNQS	- CSM Screen Source file
<product id>SCRNQC	- CSM Screen Compile Obey file
<product id>SCRNQ	- CSM Screen 'OBJECT' file

where: **<product id>** is the 2 character product identifier

For example, if the ATM TDF Requestor must be CSM'd, the CSM Screen files will be:

ATSCRNQS, ATSCRNQC & ATSCRNQ

The Requestors will then have to COPY the CSM'd screens from the appropriate Screen 'OBJECT' file.

When a screen is CSM'd, a detailed description of the modification must be included



in the CSM history of the corresponding Requestor. Also, the fact that the Screen must be compiled before the Requestor must be included in the CSM history as well as in the Installation instructions in the CSM Documentation.

4.8 Device Configuration Files

When new states or screens, which were not noted in the external spec, are added to a device Configuration file, the changes should be noted in the customer documentation.

Load groups 3000 - 3999 are reserved for CSM development.

4.9 COBNAMQS File

The COBNAMQS file will only contain the CSM'd Sections from the vanilla COBNAMES file. The appropriate COBOL module (Reports, PATHWAY Requestor/Server, etc.) will be modified to COPY the required tables from this file.

4.10 DHNAMEQS File

The DHNAMEQS file will only contain the CSM'd tables from the vanilla DHNAMES file (or tables added for the CSM). The Device Handler will be modified to retrieve the appropriate tables from this file.

4.11 Utilities

If a utility must be CSM'd, then only the procedures affected will be included in the CSM source file. The CSM source file will be placed on the appropriate subvol and the file name will follow the standard naming conventions.

For example, if a new security module must be supported for a CSM, then the following files will be located on the CUSTBA60 subvol:

- SECUTLQS - CSM'd Security Utilities Source File
- SECUTLQC - CSM'd Security Utilities Compile File
- SECUTLQ - CSM'd Security Utilities Object File

If new utilities must be developed, then it is up to the developer to adopt the appropriate naming convention. For example, if an CSM involves developing a utility



that will only be used for ATM Device Handlers, the files would be placed on the CUSTAT60 subvol and could have the following names:

- DHUTILQS - Device Handler CSM Utilities Source File
- DHUTILQC - Device Handler CSM Utilities Compile File
- DHUTILQ - Device Handler CSM Utilities Object File

4.12 Dummy Procedures

Dummy procedures are stub procedures that were inserted into vanilla source files to make CSM development easier. When dummy procedures need to be modified, all sections from the vanilla dummy procedures source file will be copied into the CSM dummy procedures source file.

4.13 LCONF Assigns and Params

If the CSM requires new assigns/params to be added to the LCONF, then the name will be prefixed by **CSM-**. All other LCONF naming conventions remain in force. A step in the installation instructions to have the customer add the new LCONF assigns/params should be added.

4.14 Log Messages

All new log message numbers added for a CSM will be in the range 10000 - 14999.

When Log Messages are added as a result of a CSM, the **Log Message Number must be unique for the specific module!!!!**

When Log Messages are added as a result of a CSM, the **Log Message Text must reflect the event initiating the message!!!!**

4.15 Token IDs

All new token IDs will begin with Q.

4.16 Field/Variable Initialization

The developer should follow accepted coding standards when initializing a field or variable. The following are some of the accepted coding standards:



If at all possible, do not use hard coded values for initialization.
Use known defines, such as `init^d` for initialization of a field or variable.
Use known functions, such as `$occurs` or `$len`, to initialize a field/variable for a length.

4.17 CSM Maintenance

CSM code must be maintained and may be built on in future CSMs. For this reason, the developer should create new procedures and use procedure calls, if the same code is required in multiple locations within a module.

5. CSM TESTING STANDARDS

5.1 CSM Unit Testing

At some point during unit testing, all CSM code should be stepped thru in inspect.

The following are a few of the things the developer should insure while in inspect:

All functionality is as stated in the External Specification Document.

Variables/fields are filled as expected, when initializations or moves of data occur.

Data does not overflow into other variables/fields, when initializations or moves of data occur.

The following should be tested by the developer for any functionality that has a limit or range of accepted values:

Upper Limit
Lower Limit
Mid-Range
Outside Upper Limit
Outside Inner Limit

Example 1: The External Specification states that the value of X, must be between 1 and 10. At a minimum, unit testing should include values for X of 10, 1, 5, 11, and 0.

Example 2: A new CSM Refresh Process is created. As Refresh can process any number of records, a test Refresh Input File, with x number of records, should be created and run thru the new Refresh Process.



The number of records in the test file should be large enough to insure the new Refresh Process can handle multiple records.

5.2 CSM Regression Testing

A CSM may add to, modify or replace product functionality. Unless specifically stated in the External Specification, product functionality should not be modified. Prior to delivering a CSM, the developer should do enough regression testing to be relatively certain that the product functionality has not been unintentionally impacted.

The following are examples of instances where product functionality was probably impacted unintentionally:

Force Post Transactions are being declined.
Auth is not able to read the CPF.

5.3 CSM Test Scripts

A Test Script should be created prior to testing, wherever feasible. The Test Script should cover Unit, Regression and System Testing. The CSM Group maintains a Microsoft Word Template that can be used to create needed Test Scripts.

6. CSM MACROS AND BUILDSKEL6

The CSM MACROS and BUILDSKEL6 are available for use by ACI Personnel only. They are not available for use by customers.

To get the CSM Macros and BUILDSKEL6, change the TACLCSTM in your default subvolume to include:

```
?TACL MACRO
== TACL created this file for your protection.

==Normal CSM Stuff
LOAD/KEEP 1/$HOME.DEBUG.DEBUGL
LOAD/KEEP 1/$DATA28.CSMNET.TACLMACS
LOAD/KEEP 1/$DATA28.BUILDSKE.CSMMACRO
ATTACHSEG SHARED $DATA28.CSMNET.CSMSEG :CSMSEG
```

\$DATA28.CSMNET.TACLMACS contains the following commands (to get this command list type "short" at your tacl prompt)



SHORT CUTS LOADED FROM \$DATA28.CSMNET.TACLMACS

E	=	EDIT <FILENAME>
F	=	FILES <SUBVOL TEMPLATE>
P	=	PERUSE
T	=	TEDIT <FILENAME>
V	=	VOLUME <VOL-NAME>
ST	=	STATUS *,TERM
SU	=	STATUS *,USER
XPAUSE	=	PAUSE
FIN	=	FILEINFO <FILE-NAME TEMPLATE>
FN	=	FILENAME <FILE-NAME TEMPLATE>
FI	=	FUP INFO <FILESET-LIST>
FID	=	FUP INFO <FILESET-LIST> , DETAIL
FS	=	FUP SUBVOLS <SUBVOLSET>
FLO	=	FUP LISTOPENS <FILESET-LIST>
LISTFREE	=	DSAP (\$DATA03,\$DATA28,\$DATA30),SHORT
CLS	=	CLEAR SCREEN
GO	=	SYNTAX = GO <NETWORK PREFIX> (EX. "GO I" GOES TO \$DATA28.QI1XCNTL)
EMS	=	SYNTAX = EMS <NETWORK PREFIX>. EX. EMS I
AFT	=	SYNTAX = AFT <NETWORK PREFIX>. EX. AFT I
NCP	=	SYNTAX = NCP <NETWORK PREFIX>. EX. NCP I
BYE	=	LOGOFF/SEGRELEASE/
HT	=	TERMINAL NAME
NETADRD	=	RUN \$SYSACI.XPNET.NETADRD / OUT \$S.#NETADRD / , ,
SUBVOLS	=	SUBVOLS <SUBVOLSET> (DISPLAYS SUBVOLS SPECIFIED IN THE SUBVOLSET)
CSMNETS	=	DISPLAY CSM NETWORKS
SHORT	=	THIS LIST

6.1 USING BUILDSKEL6

BUILDSKEL6 is available for use by ACI Personnel only. It is not available for use by customers.

BUILDSKEL6 is a TACL macro, which simplifies the generation of TAL CSM skeleton source files. The syntax for BUILDSKEL6 is as follows:

BUILDSKEL6 <vanilla_source_file_name>, <csk_skel_file_name>

The newly generated CSM skeleton source file will be generated on the developer's current volume/subvolume.



7. CSM MAKE STANDARDS

The ACI CSM MAKE Standards are contained in the CSM Make Standards 2000 1.5 Document. These standards must be followed, when developing CSMs.

8. TAL EXAMPLE CODE

```
!*FIX2.00 07/26/96 C1000S 5145 TAL AT511000 AT5A006 s **
!*****
!
! CSM History:
! -----
!
! CSM:      Description
! -----
!
! 05/30/96 CSMs to C1000QS ver. 5.1.45 NMW
! CSMPQ #99 Non-Currency Dispense
! Summary: This CSM will need to support dispensing non-
! currency items at the ATM. This includes items of value
! (phone cards, stamps, ski tickets, etc.) and zero-
! value items (coupons, marketing information, etc.).
!
! Procs Modified: ddbaea^compute^bills
!
!*****
!*****
! CSM FIX History:
! -----
!
! Date      Versn   Person SYMPTOM/PROBLEM/FIX/DEPENDENCIES/REFER
! -----
!
!*****
?nolist, source =at1000_c1000s(
? History
? Narrative
? main^street
? da^do^initialization
? get^assigns
? get^params
? daa^open^all^files
? dab^initialize^tdf
? db^initialize^timer^table
? dc^get^input
? dca^procs^timeout
? dd^procs^input^msg
? dda^procs^failed^msg
? ddaa^read^tdf
? ddb^procs^atm^input
? ddbaa^initialize^auth^msg
? ddbab^procs^track2^data
? ddbac^procs^track3^data
?list
! History
! Narrative
! main^street
! da^do^initialization
! get^assigns
! get^params
.
.
```



```
?section ddbaea^compute^bills !03005
?page "ddbaea^compute^bills" !03006
##### !03007
!# !03008
!# ddbaea^compute^bills !03009
!# !03010
!# narrative: !03011
!# This procedure will calculate the number of hi and lo bills !03012
!# which will be dispensed by the ATM for this transaction. !03013
!# !03014
!# assumptions: !03015
!# 1) TDF.hopr[0:3] contains bill values in ascending order. !03016
!# rules: !03017
!# 1) Amnt is made up from the highest bills in descending order. !03018
!# !03019c0
!# 2) Total nbr of bills must not be more than 50. #!
!# !03019c0
!# 3) We no longer check TDF..end^cash when determining bills #!!03020
!# to dispense. TDF..end^cash can go to 0 and we still allow #!!03021
!# bills to be dispensed from the hopper. #!!03022
!# !03023
!# input parameters: n/a #!!03024
!# !03025
!# output parameters: #!!03026
!# return: false - if error #!!03027
!# true - if good #!!03028
!# !03029
!# special entry conditions: n/a #!!03030
!# !03031
!# special exit conditions: n/a #!!03032
!# !03033
!# usage of globals: #!!03034
!# tdf: struct of tdf file #!!03035
!# #!!03036
##### !03037
int proc ddbaea^compute^bills; !03038
begin !03039
!03040
!03040p01
!03040p02
!03040p03
!03040p04
!03041p05
!03043c01
!03043c02
!03043c03
!03043c04
!03043c05
!03043c06
!03043c07
!03043c08
!03043c09
!03043c10
!03043c11
!03049
!03050
!03051
!03052
!03053
!03054
.
.
.
```



! Non-Currency hoppers should be skipped. These hoppers should not
! be considered for hi or lo bill value.

```
while not found do                                     !03056p01
  if ( tdf.hopr[tdf^pos].bill^val = atm.d1000.denom[atm^pos] and !03056p02
    tdf.hopr[ tdf^pos ].bill^val > 0d                        and !03056p03
    (atm.d1000.term^type = term^912^mode^d                  or !03056p04
    atm.d1000.term^type = term^912^int^mode^d )              and !03056p05
    atm.d1000.cass[ atm^pos ] <> "2"                        and !03056p06
!*****!
! BEGIN CSM #99 - Non-Currency Dispense - NMW              !
!-----!

```

```
    atm.d1000.cass[ atm^pos ] <> "9"                      and
    cur^hopper^d(tdf^pos) )                               or

```

```
!-----!
! END CSM #99 - Non-Currency Dispense - NMW                !
!*****!

```

```
  ( tdf.hopr[tdf^pos].bill^val = atm.d1000.denom[atm^pos] and !03056p07
  ( tdf.hopr[tdf^pos].bill^val = atm.d1000.denom[atm^pos] and !03056p08
  tdf.hopr[ tdf^pos ].bill^val > 0d                        and !03056p09
  (atm.d1000.term^type = term^911^mode^d                  or !03056p0A
  atm.d1000.term^type = term^911^default^d )              and !03056p0B
  atm.d1000.cass[ atm^pos ] <> "2"                        and !03056p0C
  atm.d1000.cass[ atm^pos ] <> "9" )                      or !03056p0D
  (( atm^pos := atm^pos + 1 ) > 3 ) then                  !03056p0E
    found := true                                         !03056p0F
else                                                       !03056p0G
    found := false;                                       !03056p0H
                                                         !03056p0I
                                                         !03057p0J

```

.
.
.